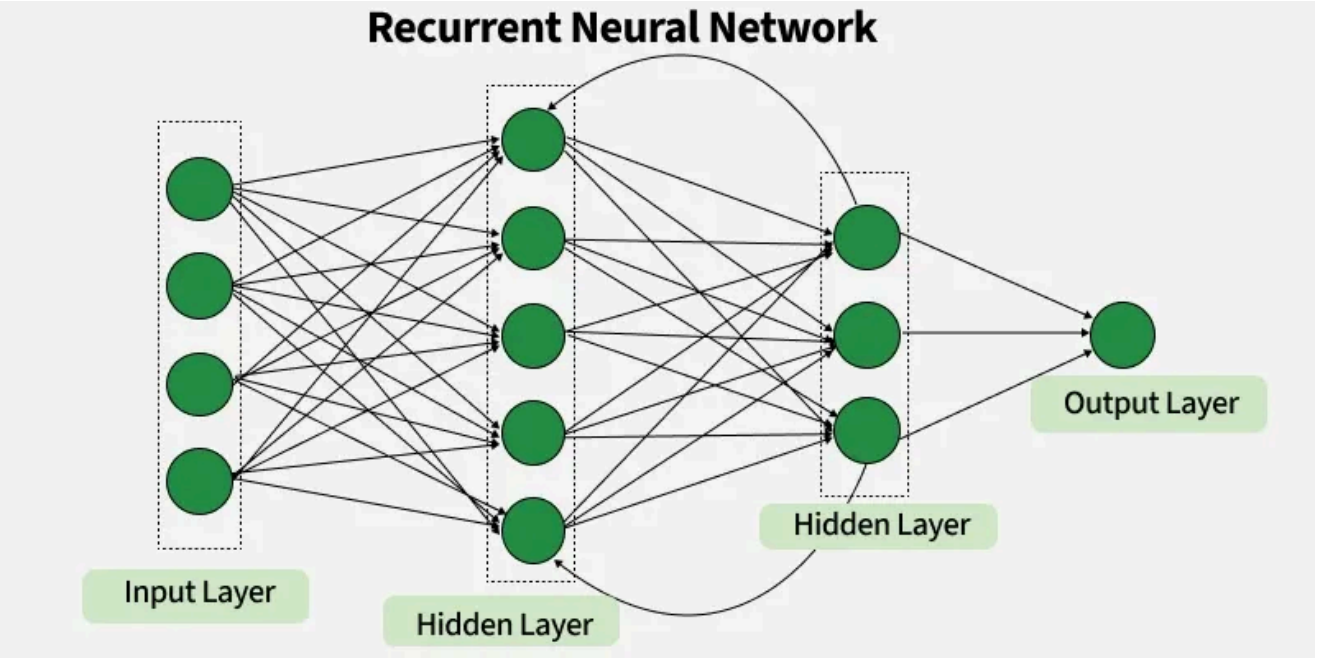
RNNs = Recurrent Neural Networks

It is a class or Neural networks designed to process sequential data by **retaining information form previous steps** (normally, DL networks assume that the input and output are independents of each other). they are especially effective for tasks where **context** and **order** matter.

- Designed for sequential and temporal data
- Maintains memory of past inputs
- Widely used in NLP, forecasting and speech tasks

It is used in:

- Translation
- text summarization
- Image captioning
- Conversational Agents
- language Modeling
- Sentiment Analysis
- language Generation

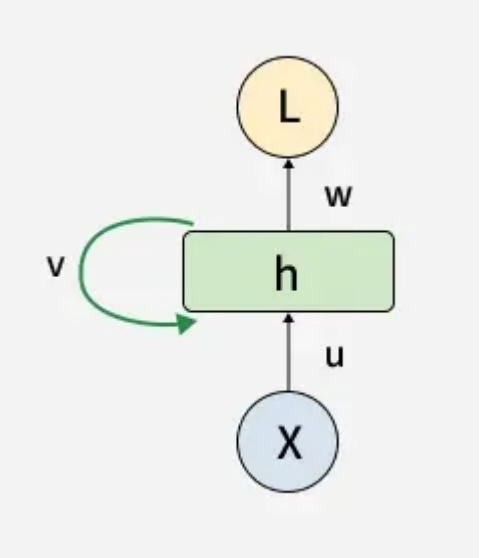


Imagine, reading a sentence and you try to predict the next word, you don't rely only on the current word but also remember the words that came before. RNNs work similarly by "remembering" past information i.e it considers all the earlier words to choose the most likely next word.
This memory of previous steps helps the network understand context and make better predictions.

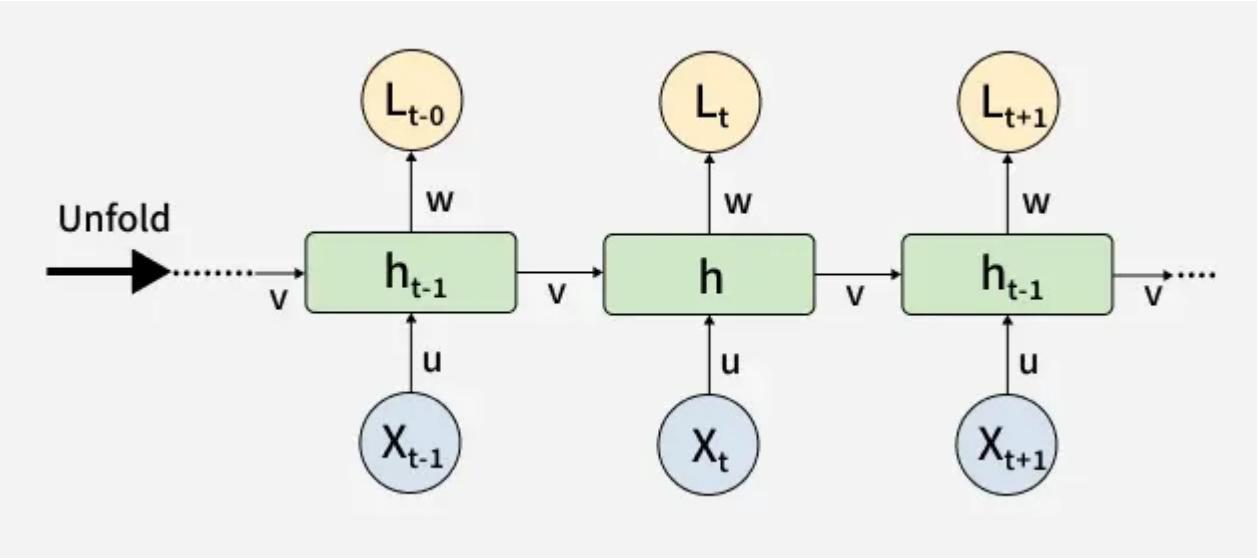
In RNNs, the input X is a sequence
RNNs is good for sequential task.
The output Y can also be a sequence (such as language translation), but often is a scalar

Two Components of RNNs

1. **Recurrent Neurons**
   The fundamental processing unit in RNN is a **Recurrent Unit.**
   they hold a [Hidden state]() that maintains information about previous inputs in a sequence.
   So the Recurrent Units can remember information from **prior steps** by feeding back their hidden state, allowing them to capture dependencies across time.



2. **RNN Unfolding**
   RNN Unfolding or Unrolling is the conceptual and visual tool used to represent the computation of a RNN over sequence of time usage of time steps. During unfolding each step of the sequence is represented as a separate later in a series illustrating how information flows across each time step.
   this unrolling enables [Back propagation]() **through time (BPTT)** a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn dependencies within sequential data.



In text generation task, RNNs track the context by maintaining a hidden state at each time step. A feedback loop is created by passing the hidden state from one-time step to the next. the **hidden state act as a memory that store information about the previous inputs**.

At each time step, the RNN processes the current input (for example, a word in a sentence) along with the hidden state from the previous time step.
This allows the RNN to 'remember' previous data points and use that information to influence the current output.